SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I **M. Sriram / 192311120** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – ANSWERS

Course: Internet Programming (CSA43) | CO-1 | Total Marks: 30

Name / Reg No: M. Sriram (192311120)

Question 1: Online Symposium Portal

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

1. Missing HTML5 semantic elements

The snippet is only a bare form. It has no page structure and no semantic elements to describe what each part of the page means. The following are missing:

- **Document skeleton:** <!DOCTYPE html>, <html lang="en">, <head> with <meta charset="UTF-8">, <meta name="viewport"> and <title>.
- **<header>:** for the symposium title/logo and introduction.
- **<nav>:** for links such as Home, Schedule and Contact.
- **<main> and <section>:** to hold the primary content and the registration area.
- **<fieldset> and <legend>:** to group the related form controls under a meaningful title.
- **<label>:** to attach a text description to every input (currently there are none).
- **<footer>:** for contact details and copyright information.

Apart from these, the inputs have no id, required, or placeholder attributes, and the button has no explicit type.

2. Modified, standards-compliant structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>National Technical Symposium 2026</h1>
    <nav>
      <a href="/">Home</a> |
      <a href="/schedule">Schedule</a> |
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Student Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant Details</legend>

          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name"
```

```
                placeholder="Enter your full name"
                autocomplete="name" required>

          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email"
                placeholder="you@example.com"
                autocomplete="email" required>

          <button type="submit">Register</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 College Symposium Committee</p>
  </footer>
</body>
</html>
```

3. Interpreting the HTTP request generated on submission

Because method="POST" is used, the browser places the form data in the body of the request, not in the URL. The data is encoded as application/x-www-form-urlencoded (the default enctype) as name=value pairs joined by &. A controlled example is:

```
POST /register HTTP/1.1
Host: www.symposium.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 44
User-Agent: Mozilla/5.0 ...
Accept: text/html

name=Sriram&email=sriram%40example.com
```

- **Request line:** method (POST), target (/register, taken from the action attribute) and protocol version.
- **Headers:** Host identifies the server, Content-Type tells the server how to parse the body, and Content-Length gives the body size.
- **Blank line:** separates the headers from the body.
- **Body:** the field values, taken from each input's name attribute. A field without a name attribute is never sent. Special characters are percent-encoded (@ becomes %40).

4. HTTP response cycle from server to browser

- The browser resolves the domain through DNS, opens a TCP connection (and a TLS handshake for HTTPS), then sends the POST request above.
- The web server routes /register to the application handler (for example PHP, Node.js, or Java).
- The handler validates and sanitises the data, then stores it in the database and may send a confirmation email.
- The server builds the HTTP response.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1250
Set-Cookie: session=abc123; HttpOnly; Secure

<html> ... Registration successful ... </html>
```

- **Status line:** 200 OK or 201 Created for success, 302/303 for a redirect to a thank-you page, 400 for invalid input, 500 for a server error.
- **Headers:** describe the content type, length, cookies, caching and security policy.
- **Body:** the HTML confirmation page.
- The browser parses the HTML, builds the DOM, fetches linked CSS, JS and images with further GET requests, and renders the page.

Good practice is the Post/Redirect/Get pattern: the server answers a successful POST with 303 See Other and the browser then fetches a confirmation page with GET. This stops a duplicate registration if the user refreshes the page.

5. Improvements for usability and accessibility

- **Labels:** each <label for> is linked to its input id, so screen readers announce the field and a click on the label focuses the input.
- **Validation:** required, type="email", pattern and minlength give instant feedback. Server-side validation must still be done as well, because client-side checks can be bypassed.
- **Helpful hints:** placeholders, autocomplete attributes and clear, specific error messages shown next to the field (use aria-live="polite" or role="alert").
- **Keyboard and focus:** logical tab order, visible :focus outlines, and a button of type="submit" so Enter submits the form.
- **Visual accessibility:** adequate colour contrast (WCAG 2.1 AA at least 4.5:1), relative font units, and responsive layout through the viewport meta tag.
- **Language and structure:** lang attribute, correct heading hierarchy (h1, then h2), and ARIA landmarks only where native semantic elements are insufficient.
- **Security and feedback:** serve over HTTPS, add a CSRF token and show a confirmation message after registration.

Question 2: Cross-Browser Compatibility Issue

```
<html>
 <head>
  <title>Company</title>
 </head>
 <body>
  <div>
   <h1>Welcome
   <p>About us
  </div>
 </body>
</html>
```

1. Improper nesting and missing closing tags

- **<h1> is never closed:** there is no </h1> after "Welcome".
- ** is never closed:** there is no after "About us".
- **Improper nesting:** because </h1> is missing, the opens while <h1> is still open, so a block-level paragraph is placed inside a heading. A heading may contain only phrasing (inline) content, so this is invalid.
- **Other omissions:** no <!DOCTYPE html>, no lang attribute on <html>, no <meta charset>, and no viewport meta tag.

2. How different browsers may interpret this structure

- HTML is forgiving. Every browser applies error-recovery rules instead of stopping, but older browsers recovered differently from each other, and that is what produces inconsistent layout.
- **Quirks mode:** without a DOCTYPE the browser falls back to quirks mode. Differences from standards mode include the box model calculation, line-height handling, table font inheritance and image alignment, so the same page measures differently.
- **Nesting differences:** modern browsers follow the HTML5 parsing algorithm. They implicitly end the open elements at , so the <p> ends up inside the <h1> and its text gets the heading's font size and weight. Older engines could close the heading early or build a different DOM tree, which gives different spacing and font sizes.
- **Default styles:** each browser has its own user-agent stylesheet, so margins and font sizes for h1 and p differ slightly.
- **Stricter parsers:** if the page were served as XHTML (application/xhtml+xml), the XML parser would reject it with a well-formedness error and show no page at all.
- **Scripts and CSS:** selectors such as div > h1 + p or DOM scripts operating on this broken tree can give different results.

3. Missing semantic elements

The generic carries no meaning. The page should use <header> for the site heading, <nav> for navigation, <main> for the primary content, <section> (or <article>) for the About Us block, and <footer> for closing information. Semantic tags improve accessibility (screen reader landmarks), SEO and maintainability, and they make browsers' behaviour more predictable.

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>

  <main>
    <section>
      <h2>About Us</h2>
      <p>About us</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company</p>
  </footer>
</body>
</html>
```

Every element is now correctly closed and nested, the heading holds only text, and the document can be checked with the W3C Markup Validation Service (validator.w3.org).

5. Techniques to ensure cross-browser compatibility

- **Always declare <!DOCTYPE html>** so that all browsers render in standards mode.
- **Validate the code** with the W3C validator and fix all nesting and closing errors.

- **Reset or normalise CSS** (for example normalize.css) to remove differences between user-agent stylesheets.
- **Use modern layout** (Flexbox, Grid, relative units, media queries) instead of table layouts or fixed pixel hacks.
- **Vendor prefixes and autoprefixer** for CSS features that still need them. Check support on caniuse.com.
- **Progressive enhancement and graceful degradation:** the basic content works everywhere and advanced features are added on top. Use feature detection (@supports, Modernizr) instead of browser sniffing.
- **Polyfills and fallbacks** for missing APIs, and use of standard semantic HTML5 elements.
- **Test on many browsers and devices** (Chrome, Firefox, Safari, Edge, mobile) using real devices, developer tools, or services such as BrowserStack.

Question 3: Form Submission Failure

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

1. Issues in the form configuration

- **Wrong button type:** type="button" is a plain push button with no submit behaviour. It is the main reason nothing is sent.
- **Wrong HTTP method:** GET is used for a login form, so the username and password would be appended to the URL.
- **No labels:** the inputs have no <label>, ids, or placeholders, which hurts accessibility and usability.
- **No validation or autocomplete hints:** no required attribute, and no autocomplete="username" / "current-password" so password managers cannot help.
- **Missing semantics and security:** no fieldset/legend, no HTTPS mention, and no CSRF token.

2. Why the HTTP request is not triggered

A form is submitted only when a submit control is activated: a <button type="submit"> (the default for <button> inside a form), an <input type="submit">, an <input type="image">, or the user pressing Enter in a field that triggers an implicit submit. Here the button is type="button", whose default action is nothing. No submit event is fired, and there is no JavaScript click handler that calls form.submit(), fetch(), or XMLHttpRequest. As a result the browser never builds a request, and the server receives no data. Nothing appears in the Network tab either.

3. Role of the GET method in this scenario

GET is designed for safe, idempotent retrieval of resources. The form data is encoded in the URL query string, for instance:

```
GET /submit?username=sriram&password=MyPass123 HTTP/1.1
Host: www.example.com
```

If the button did submit the form, this would create serious problems:

- The password is visible in the address bar, over the user's shoulder.
- It is stored in browser history, bookmarks, proxy logs and server access logs, and may leak through the Referer header.

- URLs have length limits, and responses to GET may be cached.

GET is therefore suitable for search or filter forms, but wrong for authentication. Credentials must be sent in the body of a POST request.

4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>User Login</h1>
    <form action="/submit" method="POST">
      <input type="hidden" name="csrf_token" value="{{token}}">
      <fieldset>
        <legend>Sign in</legend>

        <label for="username">Username:</label>
        <input type="text" id="username" name="username"
               autocomplete="username" required>

        <label for="password">Password:</label>
        <input type="password" id="password" name="password"
               autocomplete="current-password" minlength="8" required>

        <button type="submit">Login</button>
      </fieldset>
    </form>
  </main>
</body>
</html>
```

The two key fixes are method="POST" and type="submit". The labels, validation and hidden CSRF token are additional improvements.

5. Improvements for secure data transmission

- **Use HTTPS (TLS):** the form action must be an https:// URL so that credentials are encrypted in transit. Enable HSTS to prevent downgrade to HTTP.
- **Use POST:** never send credentials in the query string.
- **Store passwords safely:** hash with a salted, slow algorithm (bcrypt, scrypt, Argon2) on the server. Never store them in plain text.
- **CSRF protection:** anti-CSRF tokens and SameSite cookies.
- **Secure sessions:** cookies with Secure, HttpOnly and SameSite flags, and regenerate the session ID after login.
- **Server-side validation and sanitisation:** use parameterised queries to prevent SQL injection and escape output to prevent XSS.
- **Abuse protection:** rate limiting, account lockout or CAPTCHA, and optionally multi-factor authentication.
- **Safe error messages:** a generic message such as "Invalid username or password" that does not reveal which part was wrong.
- **Security headers:** Content-Security-Policy, X-Content-Type-Options, and Cache-Control: no-store on sensitive pages.